#2

Valid Anagram

Arrays & Hashing | LeetCode 242 | NeetCode "Valid Anagram"

PROBLEM

Given two strings `s` and `t`, return `true` if the two strings are anagrams of each other, otherwise return `false`. An anagram is a string that contains the exact same characters as another string, but the order of the characters can be different.

```
s = "racecar", t = "carrace" → true
```

```
s = "jar", t = "jam" → false
```

CORE IDEA

Goal

Determine whether two strings are anagrams of each other.

Key insight

Two strings are anagrams exactly when they contain the same characters the same number of times — order doesn't matter, so the problem reduces to comparing character frequency counts instead of comparing the strings directly.

> Trigger: “Same letters, different order?” → reach for a character frequency count (Hash Map).

OPTIMAL SOLUTION

Data structure — Two Hash Maps (Python `dict`) tracking character counts

1. If `s` and `t` have different lengths, they can't be anagrams — return `False` immediately.
2. Walk both strings together in one pass, building `counterS` and `counterT` with `dict.get(char, 0) + 1`.
3. For every character in `counterS`, compare its count against `counterT.get(char, 0)`; any mismatch means the strings aren't anagrams — return `False`.
4. If every character's count matches, return `True`.

O(n + m)	O(1)
TIME	SPACE

CODE

```python
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        counterS, counterT = {}, {}
        if len(s) != len(t):
            return False

        for ch in range(len(s)):
            counterS[s[ch]] = 1 + counterS.get(s[ch],
0)
            counterT[t[ch]] = 1 + counterT.get(t[ch],
0)

        for c in counterS:
            if counterS[c] != counterT.get(c, 0):
                return False

        return True
```

TIPS TO REMEMBER

Mental shortcut — “Same letters, same counts → anagram.”

Common / related — Group Anagrams, Two Sum, Contains Duplicate

> *"I'm comparing character frequency counts instead of sorting, which keeps this at O(n) time instead of O(n log n) — and because the alphabet is fixed at 26 lowercase letters, the extra space is effectively O(1)."*

EDGE CASES

- Different lengths → return False immediately, no counting needed
- All identical characters, e.g. "aaa" vs "aaa" → true
- s and t are identical strings → trivially anagrams
- Input is lowercase-only per constraints — mixed case or Unicode input would need normalizing first